

## **Type 1**

### **Package Import**

```
import java.sql.*;
```

### **Explanation**

Imports all the JDBC classes required to connect and work with the database.

### **Main Method**

```
public static void main(String[] args) throws Exception {  
    readRecords();  
    insertRecords();  
    insertVar();  
    insertUsingPst();  
    update();  
    commitdemo();  
}
```

### **Explanation**

The main() method calls all the database methods one by one.

### **1. readRecords()**

```
public static void readRecords() throws Exception
```

### **Purpose**

Reads all employee records from the database.

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/employee";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

These variables contain the database details.

- **localhost** → Database server
- **3306** → MySQL default port
- **employee** → Database name

### **SQL Query**

```
String query = "select * from emp";
```

This retrieves all records from the **emp** table.

### **Connection**

```
Connection con = DriverManager.getConnection(url, username, password);
```

Creates a connection with the MySQL database.

---

### **Statement**

```
Statement st = con.createStatement();
```

Creates a Statement object to execute SQL queries.

---

### **Execute Query**

```
ResultSet rs = st.executeQuery(query);
```

Executes the SELECT query and stores the result in a ResultSet.

---

### **Reading Records**

```
while(rs.next())
```

Moves through each row.

```
System.out.println("Name is " + rs.getString(1));
```

```
System.out.println("ID is " + rs.getInt(2));
```

```
System.out.println("Age is " + rs.getInt(3));
```

Prints

- Employee Name
  - Employee ID
  - Employee Age
- 

### **Close Connection**

```
con.close();
```

Releases the database connection.

---

## **2. insertRecords()**

public static void insertRecords() throws Exception

### **Purpose**

Inserts one employee record using **Statement**.

### **SQL Query**

String query = "insert into emp values('Siva',10,20)";

This inserts

**Name ID Age**

Siva 10 20

---

### **Execute**

```
int rows = st.executeUpdate(query);
```

executeUpdate() is used for

- INSERT
- UPDATE
- DELETE

It returns the number of affected rows.

Output

Number of Rows affected: 1

---

### **3. insertVar()**

public static void insertVar() throws Exception

#### **Purpose**

Inserts values stored in Java variables.

#### **Variables**

```
int id = 9;
```

```
String name = "Kavya";
```

```
int age = 20;
```

### **SQL Query**

String query =

```
"insert into emp values("
```

```
+ name + ", "
```

```
+ id + ", "
```

```
+ age + ")";
```

Generated query becomes

```
insert into emp values('Kavya',9,20);
```

---

### **Execute**

```
st.executeUpdate(query);
```

Adds the record to the database.

---

### **Advantage**

Values can be changed dynamically.

### **Disadvantage**

Uses string concatenation, making it vulnerable to **SQL Injection**.

---

## **4. insertUsingPst()**

```
public static void insertUsingPst() throws Exception
```

### **Purpose**

Inserts records using **PreparedStatement**.

---

### **SQL Query**

```
String query = "insert into emp values(?,?,?)";
```

“?” represents placeholders.

---

### **Setting Values**

```
pst.setString(1,name);
```

```
pst.setInt(2,id);
```

```
pst.setInt(3,age);
```

### **Execute**

```
pst.executeUpdate();
```

### **Inserts**

| Name | ID | Age |
|------|----|-----|
|------|----|-----|

|          |   |    |
|----------|---|----|
| PorSelvi | 7 | 28 |
|----------|---|----|

---

### **Advantages**

- Prevents SQL Injection

- Faster
  - Recommended method
  - Easy to reuse
- 

### **5. update()**

public static void update() throws Exception

#### **Purpose**

Updates the age of an employee.

#### **SQL Query**

update emp

set emp\_age = 24

where emp\_id = 5;

Changes

Employee ID = 5

Age becomes

24

---

#### **Execute**

st.executeUpdate(query);

Returns the number of updated rows.

---

### **6. commitdemo()**

public static void commitdemo() throws Exception

#### **Purpose**

Demonstrates **Transaction Management**.

---

#### **Auto Commit Disabled**

con.setAutoCommit(false);

Normally MySQL saves every query immediately.

This statement stops automatic saving.

---

#### **First Query**

```
update emp
set emp_age = 20
where emp_id = 5;
```

---

### Second Query

```
update emp
set emp_age = 22
where emp_id = 6;
```

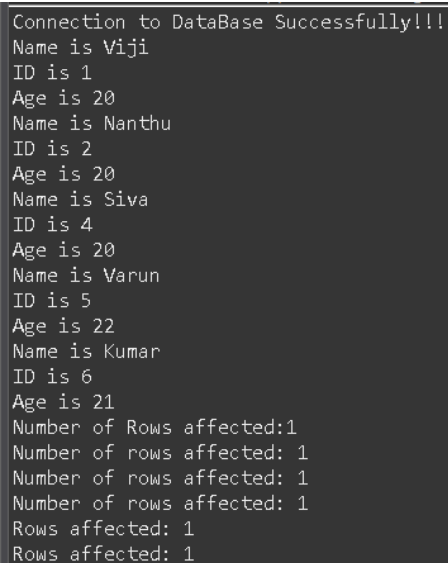
---

### Commit

```
con.commit();
```

Both updates become permanent together.

### Result:



```
Connection to DataBase Successfully!!!
Name is Viji
ID is 1
Age is 20
Name is Nanthu
ID is 2
Age is 20
Name is Siva
ID is 4
Age is 20
Name is Varun
ID is 5
Age is 22
Name is Kumar
ID is 6
Age is 21
Number of Rows affected:1
Number of rows affected: 1
Number of rows affected: 1
Number of rows affected: 1
Rows affected: 1
Rows affected: 1
```

### Type 2

#### Package Import

```
import java.sql.*;
```

#### **Explanation**

Imports all the JDBC classes required to connect and work with the MySQL database.

---

## **Class 1 : Main1**

```
public class Main1 {  
    public static void main(String[] args) throws Exception
```

### **Purpose**

Establishes a connection between the Java application and the MySQL database.

---

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/school";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

These variables contain the database details.

- **localhost** → Database server
  - **3306** → MySQL default port
  - **school** → Database name
- 

### **Connection**

```
Connection con = DriverManager.getConnection(url, username, password);
```

Creates a connection with the MySQL database.

---

### **Output**

```
System.out.println("Connected to DataBase Successfully");
```

Displays the success message after establishing the connection.

---

### **Close Connection**

```
con.close();
```

Releases the database connection.

### **Result:**

```
Connected to DataBase Successfully
```

---

## **Class 2 : Insert**

```
public class Insert {
```

```
public static void main(String[] args) throws Exception
```

### **Purpose**

Inserts a new record into the **student** table.

---

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/school";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

Connects to the **school** database.

---

### **SQL Query**

```
String query = "insert into student(std_name) values('Viji')";
```

This inserts

#### **Student Name**

Viji

---

### **Statement**

```
Statement stmt = con.createStatement();
```

Creates a Statement object to execute SQL queries.

---

### **Execute**

```
stmt.executeUpdate(query);
```

executeUpdate() is used for

- INSERT
- UPDATE
- DELETE

It returns the number of affected rows.

---

### **Output**

```
System.out.println("Record Inserted Successfully");
```

Displays the success message after inserting the record.

---



### **Close Connection**

```
con.close();
```

Releases the database connection.

### **Result:**

```
Record Inserted Successfully
```

---

### **Class 3 : Delete**

```
public class Delete {  
    public static void main(String[] args) throws Exception
```

### **Purpose**

Deletes an existing record from the **student** table.

---

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/school";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

Connects to the database.

---

### **SQL Query**

```
String query = "delete from student where std_name='Viji'";
```

Deletes the student whose name is **Viji**.

---

### **Statement**

```
Statement stmt = con.createStatement();
```

Creates a Statement object.

---

### **Execute**

```
stmt.executeUpdate(query);
```

Deletes the record from the table.

---

### **Output**

```
System.out.println("Record Deleted Successfully");
```

Displays the success message.

---

### **Close Connection**

```
con.close();
```

Releases the database connection.

### **Result:**

```
Record Deleted Successfully
```

---

### **Class 4 : Select**

```
public class Select {  
    public static void main(String[] args) throws Exception
```

### **Purpose**

Retrieves all student records from the database.

---

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/school";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

Connects to the **school** database.

---

### **Statement**

```
Statement stmt = con.createStatement();
```

Creates a Statement object.

---

### **SQL Query**

```
ResultSet rs = stmt.executeQuery("select * from student");
```

Executes the SELECT query and stores the result in a ResultSet.

---

### **Reading Records**

```
while(rs.next())
```

Moves through each row of the table.

```
System.out.println(
```

```
rs.getString(1)+" "  
+rs.getInt(2)+" "  
+rs.getInt(3)+" "  
+rs.getString(4));
```

Prints

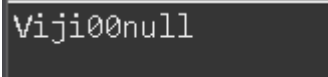
- Student Name
  - Roll Number
  - Student Age
  - Department
- 

### **Close Connection**

```
con.close();
```

Releases the database connection.

### **Result:**



```
Viji00null
```

---

### **Class 5 : Update**

```
public class Update {  
    public static void main(String[] args) throws Exception
```

### **Purpose**

Updates the age of a student.

---

### **Database Connection**

```
String url = "jdbc:mysql://localhost:3306/school";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

Connects to the **school** database.

---

### **SQL Query**

```
String query =
```

```
"update student set std_age = 20 where std_name='Viji'";
```

Changes

## Student Name New Age

Viji                      20

---

### Statement

```
Statement stmt = con.createStatement();
```

Creates a Statement object.

---

### Execute

```
stmt.executeUpdate(query);
```

Updates the student record.

---

### Output

```
System.out.println("Record Updated Successfully");
```

Displays the success message.

---

### Close Connection

```
con.close();
```

Releases the database connection.

### Result:

```
Record Updated Successfully
```

## Type 3

### Package Import

```
import java.sql.*;
```

```
import java.util.Scanner;
```

### Explanation

- `java.sql.*` → Imports JDBC classes.
  - `java.util.Scanner` → Used to get user input.
- 

### Main Method

```
public static void main(String[] args) throws SQLException
```

### Explanation

The main() method performs all database operations in sequence:

1. Connect to database
2. Create table
3. Insert hardcoded data
4. Insert user data
5. Update record
6. Alter table
7. Commit transaction
8. Delete record
9. Display records

---

## **1. Database Connection**

### **Purpose**

Connects Java to the MySQL database.

### **Code**

```
String url = "jdbc:mysql://localhost:3306/entrybook";
```

```
String username = "root";
```

```
String password = "Viji@2005";
```

```
Connection con = DriverManager.getConnection(url, username, password);
```

### **Output**

Connection Success

---

## **2. Disable Auto Commit**

### **Purpose**

Disables automatic saving of changes.

### **Code**

```
con.setAutoCommit(false);
```

---

## **3. Create Table**

### **Purpose**

Creates a table named details.

### **SQL Query**

String createTable =

```
"CREATE TABLE details(name VARCHAR(30), address VARCHAR(50), city VARCHAR(30), state VARCHAR(30), country VARCHAR(30));"
```

### **Execute**

```
stmt.executeUpdate(createTable);
```

### **Output**

Table Created

---

## **4. Insert Hardcoded Data**

### **Purpose**

Inserts a fixed record.

### **SQL Query**

String insertFixed =

```
"INSERT INTO details VALUES('John','Main Street','New York','NY','USA');"
```

### **Execute**

```
stmt.executeUpdate(insertFixed);
```

### **Output**

Hardcoded Data Inserted

---

## **5. User Input**

### **Purpose**

Accepts user details.

### **Code**

```
Scanner sc = new Scanner(System.in);
```

Inputs:

- Name
  - Address
  - City
  - State
  - Country
- 

## **6. Insert User Data**

**Purpose**

Inserts user data using PreparedStatement.

**SQL Query**

String insertUser =

"INSERT INTO details VALUES(?,?,?,?)";

**Set Values**

ps.setString(1, name);

ps.setString(2, address);

ps.setString(3, city);

ps.setString(4, state);

ps.setString(5, country);

**Execute**

ps.executeUpdate();

**Output**

User Data Inserted

---

**7. Update Record****Purpose**

Updates John's city.

**SQL Query**

String updateQuery =

"UPDATE details SET city='Chennai' WHERE name='John'";

**Execute**

stmt.executeUpdate(updateQuery);

**Output**

Data Updated

---

**8. Alter Table****Purpose**

Adds a new column.

**SQL Query**

String alterQuery =

```
"ALTER TABLE details ADD email VARCHAR(40);"
```

**Execute**

```
stmt.executeUpdate(alterQuery);
```

**Output**

New Column Added

---

**9. Commit Transaction****Purpose**

Saves all changes permanently.

**Code**

```
con.commit();
```

**Output**

Transaction Committed

---

**10. Delete Record****Purpose**

Deletes John's record.

**SQL Query**

String deleteQuery =

```
"DELETE FROM details WHERE name='John'";
```

**Execute**

```
stmt.executeUpdate(deleteQuery);
```

**Output**

One Row Deleted

---

**11. Display Records****Purpose**

Displays all remaining records.

**SQL Query**

```
ResultSet rs = stmt.executeQuery("SELECT * FROM details");
```

**Read Records**

```
while(rs.next())
```



```
System.out.println(
rs.getString(1)+" "+
rs.getString(2)+" "+
rs.getString(3)+" "+
rs.getString(4)+" "+
rs.getString(5));
```

---

## **12. Close Resources**

### **Purpose**

Closes all opened resources.

### **Code**

```
rs.close();
ps.close();
stmt.close();
sc.close();
con.close();
```

### **Result:**

```
Connection Success
Table Created
Hardcoded Data Inserted
Enter Name:
Vijayalakshmi M
Enter Address:
Thoraipakkam
Enter City:
Chennai
Enter State:
Tamil Nadu
Enter Country:
India
User Data Inserted
Data Updated
New Column Added
Transaction Committed
One Row Deleted

Final Table Data:
Vijayalakshmi M Thoraipakkam Chennai Tamil Nadu India
```